WGUPS Routing Program

Algorithm and Data Structure Planning Document

A. Algorithm Identification

To efficiently route the delivery trucks and ensure all WGUPS constraints are met (including traveling under 140 miles total), a **Greedy Algorithm**—specifically the **Nearest Neighbor Algorithm**—will be implemented. This is a self-adjusting heuristic approach commonly used to approximate solutions for the Traveling Salesperson Problem (TSP). The algorithm is "self-adjusting" in this context because at each node (delivery location), it dynamically evaluates the remaining unvisited package locations and consistently chooses the immediately closest valid destination from its current point. This approach is highly appropriate as it optimizes localized route decisions to keep overall mileage low.

B. Data Structure Identification

To store, retrieve, and manage the package data effectively, a **Chaining Hash Table** (a self-adjusting data structure) will be utilized. Rather than relying on Python's built-in dictionaries, a custom hash map will be built from scratch to demonstrate core competency, managing collisions by chaining elements within the same bucket using internal lists.

1. Explanation of Data Structure

The Chaining Hash Table accounts for the relationship between the data components by mapping a single unique key—the Package ID—to a comprehensive `Package` object (the value). The `Package` object encapsulates all related attributes of a delivery component: delivery address, deadline, city, zip code, weight, special notes, and current delivery status (e.g., *At the Hub, En Route, Delivered at 10:30 AM*). By hashing the Package ID, the data structure provides a direct mathematical mapping to the specific memory bucket containing that parcel's details. This allows the routing program to instantly retrieve and update a package's address (such as correcting Package #9 at 10:20 a.m.) or status without executing a linear search through all packages.

C. Program Overview

1. Algorithm’s Logic (Pseudocode)

```
INITIALIZE Hash_Table with a capacity of 40 buckets
LOAD package details from CSV into Hash_Table using Package ID as key

CLASS Truck:
    current_location = "Hub"
    time = 08:00:00
    packages_loaded = [list of package IDs]
    mileage = 0.0

FUNCTION Nearest_Neighbor_Route(Truck):
    WHILE Truck.packages_loaded IS NOT EMPTY:
        min_distance = INFINITY
        next_package = NULL

        // Evaluate all remaining packages on the truck
        FOR EACH pkg_id IN Truck.packages_loaded:
            package = Hash_Table.lookup(pkg_id)
            distance = lookup_distance_in_graph(Truck.current_location, package.address)

            IF distance < min_distance:
                min_distance = distance
                next_package = package

        // Update total metrics for the truck
        Truck.mileage += min_distance
        Truck.time += calculate_time(min_distance, speed=18)
        Truck.current_location = next_package.address

        // Check constraint: Handle the address update for Package 9
        IF Truck.time >= 10:20:00 AND Hash_Table.lookup(9).address == "Wrong Address":
            Hash_Table.lookup(9).address = "410 S. State St., Salt Lake City, UT 84111"

        // Finalize delivery
        next_package.status = "Delivered at " + Truck.time
        REMOVE next_package.ID FROM Truck.packages_loaded

    RETURN Truck.mileage
```

2. Development Environment

The Python application will be developed using a robust, modern programming environment to ensure reliability and performance:

- **Hardware:** A workstation equipped with an Intel Core i7 processor (or equivalent AMD Ryzen), 16 GB of RAM, and a Solid State Drive (SSD), operating on Windows 11. This hardware offers more than sufficient computational power to execute the algorithm instantly and handle overhead during debugging.
- **Software:** The program will be written in **Python 3.10**. The Integrated Development Environment (IDE) chosen is **PyCharm Community Edition** (or optionally Visual Studio Code). PyCharm provides essential tools like advanced step-through debugging, syntax highlighting, and object inspection, which are critical when monitoring state changes in the hash table and routing loops.

3. Space-Time Complexity Using Big-O Notation

The complexity of the program is evaluated across its major segments:

- **Loading Data into Hash Table:**
  Time Complexity: $O(N)$, where N is the number of packages (40). Each insertion operates in average $O(1)$ time.
  Space Complexity: $O(N)$ to store the packages and the underlying buckets.
- **Nearest Neighbor Routing Algorithm:**
  Time Complexity: $O(N^2)$ per truck route. To find the next closest package, the algorithm loops through the remaining N packages, then $N-1$, then $N-2$, resulting in a quadratic time series sum.
  Space Complexity: $O(1)$ auxiliary space, as all routing distance comparisons and time updates happen in-place via existing variables.
- **Data Lookup & Status Updates:**
  Time Complexity: $O(1)$ average case for querying the hash table to report statuses.
  Space Complexity: $O(1)$.
- **Overall Program Complexity:**
  Total Time Complexity: $O(N^2)$, bounded by the nested routing logic.
  Total Space Complexity: $O(N)$, bounded by the space required to maintain the map and package hash table data in memory.

4. Scalability and Adaptability

The proposed design balances efficiency with the constraints of the scenario, offering moderate scalability. A Chaining Hash Table scales gracefully; if the package volume grows significantly, the table dynamically resizes (increasing its bucket count and rehashing keys) to maintain its $O(1)$ average lookup time, ensuring no lag when searching for package details.

The Nearest Neighbor algorithm is well-suited for localized delivery routes with smaller datasets. However, because its time complexity is $O(N^2)$, extreme scaling (e.g., jumping from 40 to 40,000 packages) would cause notable processing delays. To adapt to massive growth, the software could cluster deliveries using K-Means or partition the city into grid zones first, subsequently running the Nearest Neighbor algorithm strictly within smaller zones to keep N low for any single calculation.

5. Software Efficiency and Maintainability

The application is designed using Object-Oriented Programming (OOP) principles, separating logic into self-contained entities: `Package`, `Truck`, and `ChainingHashTable` classes. This architecture makes the code both efficient to execute and exceptionally easy to maintain.

If WGUPS alters business parameters—such as acquiring a truck that holds 25 packages instead of 16, or hiring an extra driver—these variables can simply be adjusted within the `Truck` instantiation without rewriting the routing logic. Furthermore, isolating location data allows WGU to deploy this exact software to different cities simply by replacing the input CSV files containing new addresses and distances, requiring zero changes to the underlying source code.

6. Strengths and Weaknesses of the Self-Adjusting Hash Table

- **Strengths:** The primary advantage is its exceptional speed. It delivers $O(1)$ average-case time complexity for data insertions, updates, and lookups. This enables instant on-demand status reporting for supervisors and swift address corrections for anomalies like Package #9. The chaining strategy inherently handles dynamic resizing by appending collisions into a list, adjusting to new data naturally.
- **Weaknesses:** A major drawback of a hash table is that it fundamentally stores unordered data. If WGUPS needs a sorted manifest (e.g., all packages arranged by delivery deadline or weight), the hash table cannot provide it directly. Extracting and sorting elements costs extra time, $O(N \log N)$. Additionally, poorly designed hash functions lead to severe

collisions, clustering data in one bucket and degrading lookup performance to ***O(N)***. It also consumes slightly more memory space than standard arrays due to empty bucket overhead and linked list pointers.

7. Data Key Justification

The most appropriate key choice for efficient delivery management is the **Package ID**.

For a hash table to function efficiently and avoid immense collision rates, the chosen key must be strictly unique. Attributes such as *delivery address*, *city*, or *zip code* are inherently non-unique, as multiple parcels are frequently destined for the same office or neighborhood. Attributes like *delivery deadline* or *package weight* also overlap frequently across different parcels. *Delivery status* is mutable and will constantly change, invalidating it as a static key.

The *Package ID* is guaranteed to be a singular, unique integer for every parcel. Utilizing it as the primary key minimizes hash collisions, prevents accidental data overwrites when updating delivery times, and naturally maps to how a supervisor or customer queries the system (e.g., "What is the status of Package #9?").

D. Sources

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Python Software Foundation. (n.d.). *Data Structures — Python 3.10.x documentation*. Retrieved from https://docs.python.org/3/tutorial/datastructures.html
- ZyBooks. (n.d.). *Data Structures and Algorithms in Python*. Interactive online textbook.